



Lab: Administración del almacenamiento



Información del Laboratorio

Atributo	Descripción
Dificultad	Intermedio
Tiempo Estimado	45 minutos
Servicios Principales	EC2, EBS, S3, IAM, AWS CLI
Requisitos Previos	Cuenta AWS con acceso a consola, nociones básicas de EC2 y Linux



Resumen y Objetivos



Resumen

Este laboratorio te guía en la administración de almacenamiento AWS mediante AWS CLI y recursos EC2. Crearás snapshots de volúmenes EBS, programarás automatizaciones con cron, y sincronizarás datos locales con un bucket Amazon S3 usando versioning para recuperar archivos eliminados.



Objetivos de Aprendizaje

Al completar este laboratorio, serás capaz de:

- 📷 **Crear y mantener** snapshots de volúmenes EBS para instancias EC2
- 🛠️ **Configurar recursos** S3 e IAM para integración con instancias EC2
- ⌚ **Programar** creación recurrente de snapshots con cron
- 🛠️ **Mantener** un historial de snapshots conservando únicamente los más recientes
- ☁️ **Sincronizar** datos desde un volumen EBS a un bucket S3 con `aws s3 sync`
- 🔄 **Recuperar** archivos eliminados en S3 mediante versioning

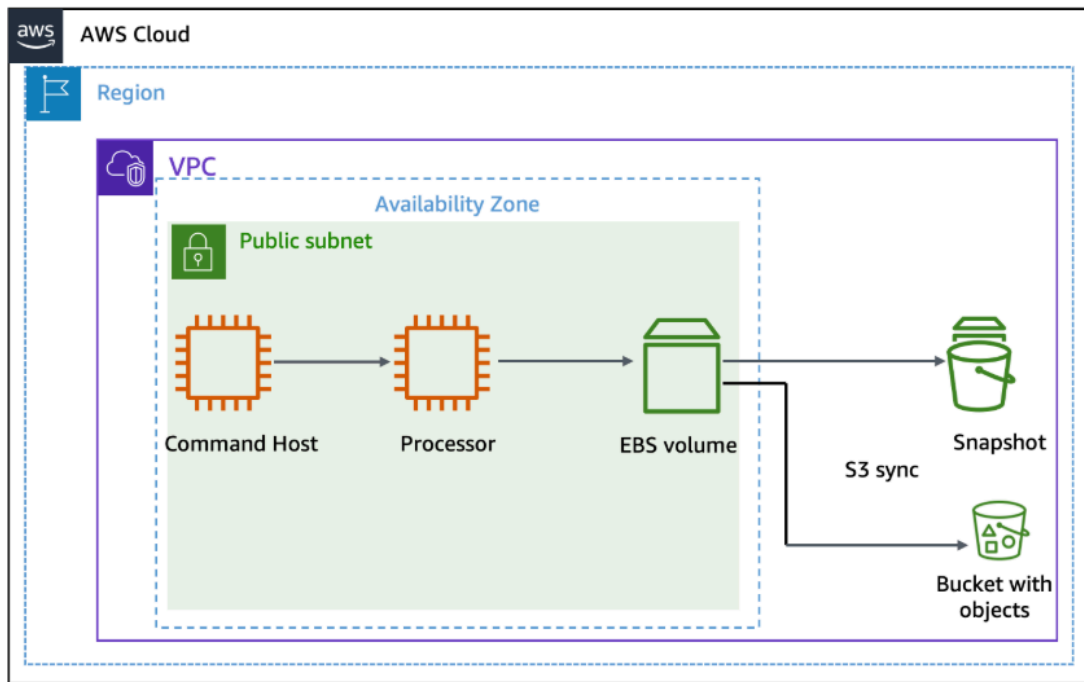


Análisis del Escenario



Contexto Inicial

El entorno incluye una VPC con un subnet público y dos instancias EC2: **Command Host** y **Processor**. La instancia **Command Host** se utilizará para administrar recursos y ejecutar comandos AWS CLI, mientras que **Processor** contiene el volumen EBS de datos.



Solicitud Implícita

Se necesita establecer un flujo de trabajo de respaldo y recuperación para EBS, automatizar la retención de snapshots y sincronizar datos de un volumen local a un bucket S3 con capacidad de versionado.

Desafío Técnico

Implementar un proceso seguro y automatizado que abarque:

- creación de bucket S3
- enlace de roles IAM
- uso de AWS CLI para snapshots y sincronización
- uso de cron para automatización temporal
- restauración de objetos con versioning

Arquitectura de la Solución

La solución es un flujo de tres capas:

1. **Command Host:** interfaz de administración AWS CLI
2. **Processor:** instancia con volumen EBS que contiene datos a respaldar y sincronizar
3. **S3 Bucket:** repositorio de archivos sincronizados con versioning habilitado

El pipeline incluye:

- snapshot EBS desde Command Host
- control de retención con Python
- sincronización de datos con `aws s3 sync`
- recuperación de versiones con `aws s3api`

Desarrollo

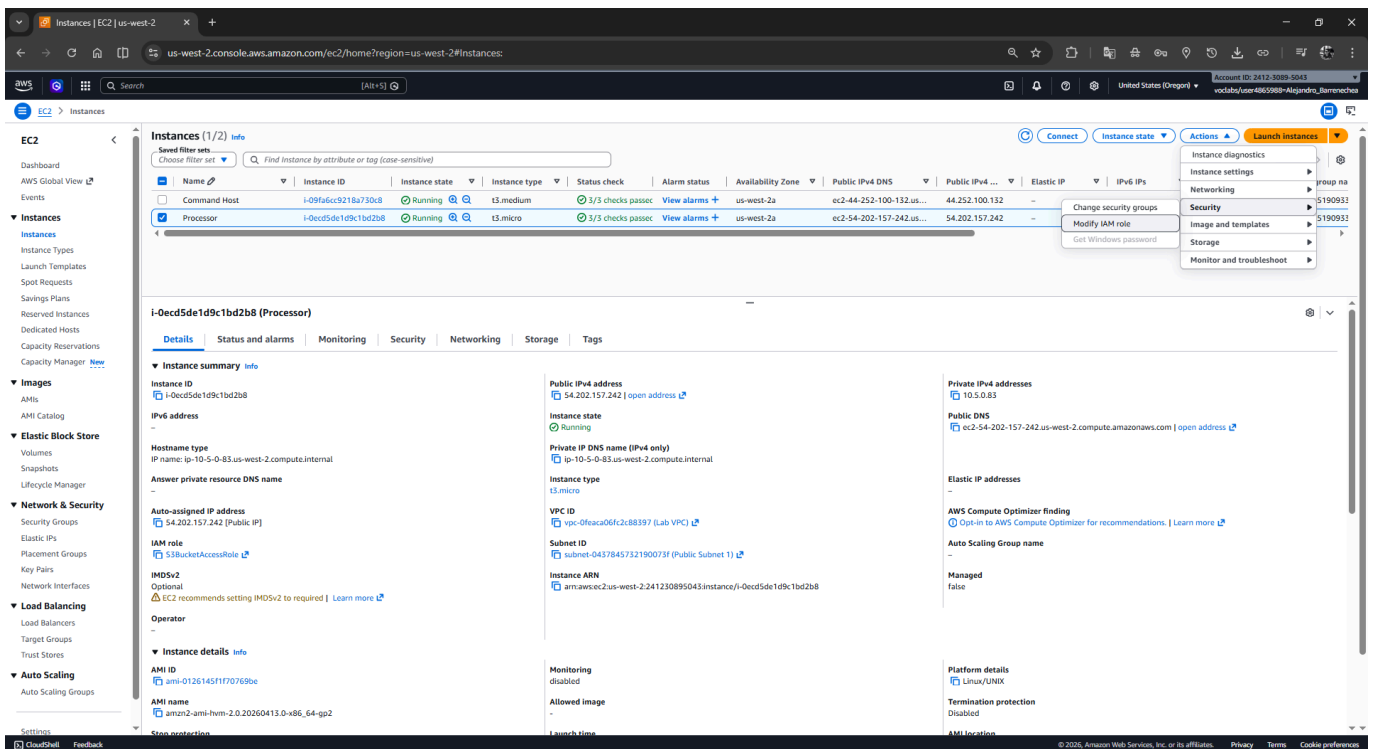
Tarea 1: Crear y configurar recursos

◆ Paso 1.1: Crear un bucket S3

1. En la barra de búsqueda de AWS Console, escribe **S3** y selecciona **S3**.
2. Elige **Create bucket**.
3. En **Bucket name** ingresa el nombre real del bucket: **lab-183-abg**.
 - Este valor será referenciado como **lab-183-abg**.
4. Deja **Region** en el valor por defecto.
5. Haz clic en **Create bucket**.

◆ Paso 1.2: Adjuntar el perfil de instancia al Processor

1. En la barra de búsqueda, escribe **EC2** y elige **EC2**.
2. En el panel izquierdo, haz clic en **Instances**.
3. Selecciona la instancia **Processor**.
4. Elige **Actions > Security > Modify IAM role**.
5. En el desplegable **IAM role**, selecciona **S3BucketAccess**.
6. Haz clic en **Update IAM role**.



The screenshot displays the AWS Management Console interface for the EC2 service. The left-hand navigation pane shows the 'Instances' section selected. The main content area shows a list of two EC2 instances. The 'Processor' instance is highlighted, and the 'Actions' menu is open, showing the 'Modify IAM role' option. The 'Details' tab for the 'Processor' instance is visible, showing its configuration including the IAM role 'S3BucketAccessRole'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
Command Host	i-09f6cc9218a730d8	Running	t3.medium	5/3 checks passed	View alarms	us-west-2a	ec2-44-252-100-132.us...	44.252.100.132	-	-
Processor	i-0ecd5de1d9c1bd2b8	Running	t3.micro	5/3 checks passed	View alarms	us-west-2a	ec2-54-202-157-242.us...	54.202.157.242	-	-

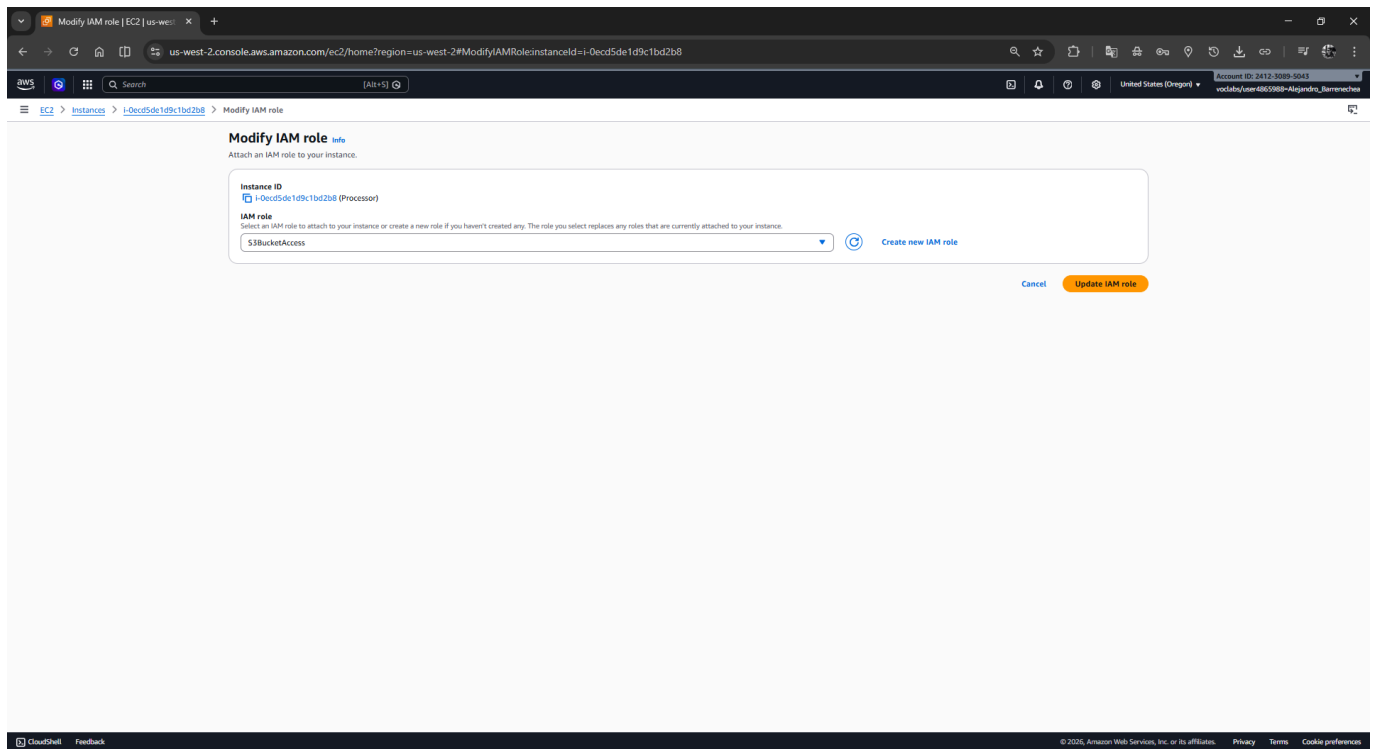
I-0ecd5de1d9c1bd2b8 (Processor)

Instance summary

Instance ID: i-0ecd5de1d9c1bd2b8
 IP v6 address: -
 Hostname type: IP name: ip-10-5-0-83.us-west-2.compute.internal
 Answer private resource DNS name: -
 Auto-assigned IP address: 54.202.157.242 [Public IP]
 IAM role: S3BucketAccessRole
 IMDSv2: Optional
 EC2 recommends setting IMDSv2 to required | Learn more

Instance details

AMI ID: ami-0126145f1f70769be
 AMI name: amzn2-ami-hvm-2.0.20260413.0-x86_64-gp2
 Monitoring: disabled
 Allowed image: -
 Platform details: Linux/UNIX
 Termination protection: Disabled



Tarea 2: Tomar snapshots de tu instancia

◆ Paso 2.1: Conectar al Command Host

1. En la consola, ve a **EC2 > Instances**.
2. Selecciona **Command Host**.
3. Haz clic en **Connect**.
4. En la pestaña **EC2 Instance Connect**, elige **Connect**.
5. Mantén abierta la terminal en la nueva pestaña.

◆ Paso 2.2: Tomar un snapshot inicial

1. Identifica el volume-id del Processor:

```
aws ec2 describe-instances --filter 'Name=tag:Name,Values=Processor' --query 'Reservations[0].Instances[0].BlockDeviceMappings[0].Ebs.{VolumeId:VolumeId}'
```

2. Identifica el instance-id del Processor:

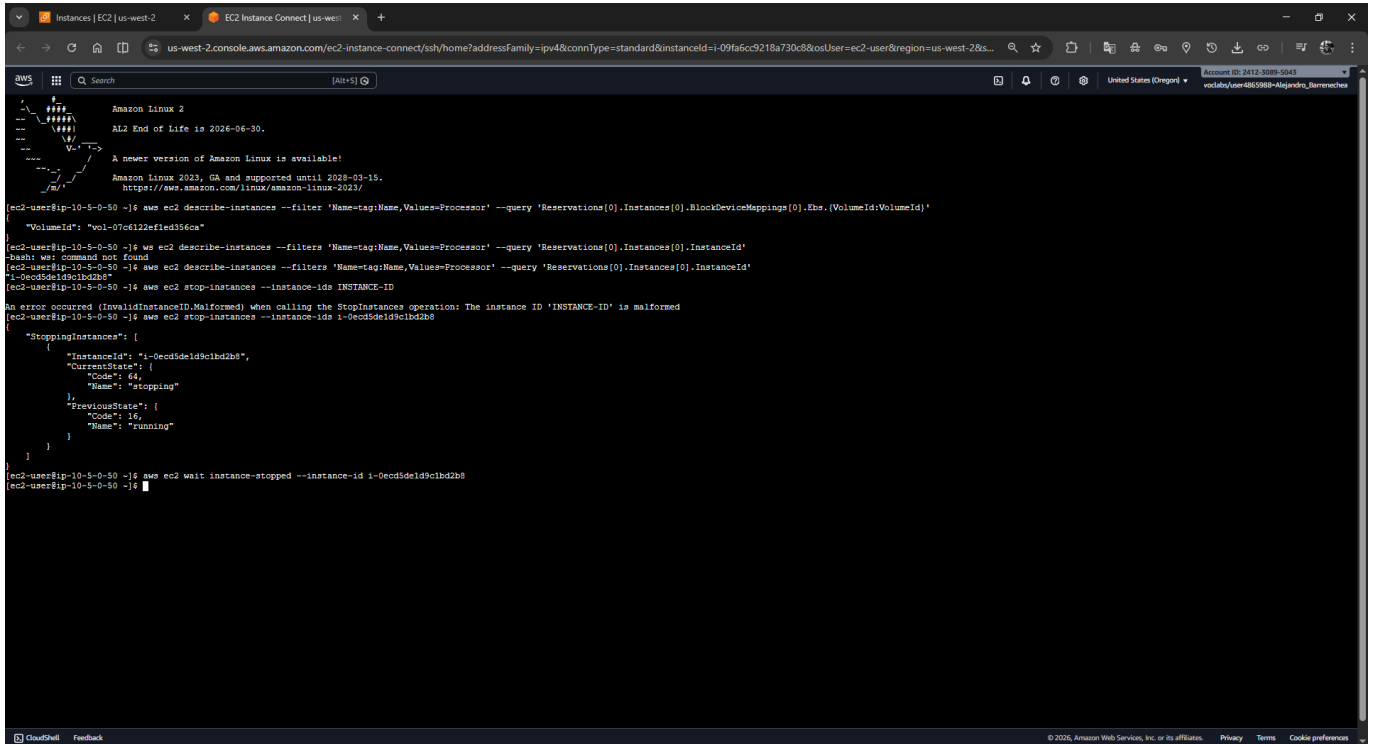
```
aws ec2 describe-instances --filters 'Name=tag:Name,Values=Processor' --query 'Reservations[0].Instances[0].InstanceId'
```

3. Detén la instancia Processor:

```
aws ec2 stop-instances --instance-ids INSTANCE-ID
```

4. Verifica que se detuvo:

```
aws ec2 wait instance-stopped --instance-id i-0ecd5de1d9c1bd2b8
```



```
Amazon Linux 2
AL2 End of Life is 2026-06-30.
A newer version of Amazon Linux is available!
Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-5-0-50 ~]$ aws ec2 describe-instances --filter 'Name=tag:Name,Values=Processor' --query 'Reservations[0].Instances[0].BlockDeviceMappings[0].Ebs.[VolumeId:VolumeId]'
{
  "VolumeId": "vol-07c6122ef1ed356ca"
}
[ec2-user@ip-10-5-0-50 ~]$ aws ec2 describe-instances --filters 'Name=tag:Name,Values=Processor' --query 'Reservations[0].Instances[0].InstanceId'
-bash: aws: command not found
[ec2-user@ip-10-5-0-50 ~]$ aws ec2 describe-instances --filters 'Name=tag:Name,Values=Processor' --query 'Reservations[0].Instances[0].InstanceId'
i-0ecd5de1d9c1bd2b8
[ec2-user@ip-10-5-0-50 ~]$ aws ec2 stop-instances --instance-ids INSTANCE-ID
An error occurred (InvalidInstanceId.Malformed) when calling the StopInstances operation: The instance ID 'INSTANCE-ID' is malformed
[ec2-user@ip-10-5-0-50 ~]$ aws ec2 stop-instances --instance-ids i-0ecd5de1d9c1bd2b8
{
  "StoppingInstances": [
    {
      "InstanceId": "i-0ecd5de1d9c1bd2b8",
      "CurrentState": {
        "Code": 44,
        "Name": "stopping"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
[ec2-user@ip-10-5-0-50 ~]$ aws ec2 wait instance-stopped --instance-id i-0ecd5de1d9c1bd2b8
[ec2-user@ip-10-5-0-50 ~]$
```

5. Crea el snapshot del volumen:

```
aws ec2 create-snapshot --volume-id vol-07c6122ef1ed356ca
```

6. Espera a que finalice:

```
aws ec2 wait snapshot-completed --snapshot-id SNAPSHOT-ID
```

7. Reinicia la instancia Processor:

```
aws ec2 start-instances --instance-ids i-0ecd5de1d9c1bd2b8
```

```

ec2-user@ip-10-5-0-50 ~$ aws ec2 wait instance-stopped --instance-id i-0e05de1d9c1bd2b8
ec2-user@ip-10-5-0-50 ~$ aws ec2 create-snapshot --volume-id vol-07c6122ef1ed356ca
{
  "Tags": [],
  "SnapshotId": "snap-0ffaadd23a04c4344",
  "VolumeId": "vol-07c6122ef1ed356ca",
  "State": "pending",
  "StartTime": "2026-04-14T23:33:53.402Z",
  "Progress": 0,
  "OwnerId": "241230895043",
  "Description": "",
  "VolumeSize": 8,
  "Encrypted": false
}
ec2-user@ip-10-5-0-50 ~$ aws ec2 start-instances --instance-ids i-0e05de1d9c1bd2b8
{
  "StartingInstances": [
    {
      "InstanceId": "i-0e05de1d9c1bd2b8",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
ec2-user@ip-10-5-0-50 ~$

```

Tarea 2.3: Programar creación recurrente de snapshots

◆ Paso 2.3.1: Crear el cron job

1. En la terminal del Command Host, crea un archivo de cron:

```
echo "* * * * * aws ec2 create-snapshot --volume-id vol-07c6122ef1ed356ca 2>&1 >> /tmp/cronlog" > cronjob
```

2. Instala el cron job:

```
crontab cronjob
```

◆ Paso 2.3.2: Verificar la creación continua

1. Ejecuta:

```
aws ec2 describe-snapshots --filters "Name=volume-id,Values=vol-07c6122ef1ed356ca"
```

```
ec2-user@ip-10-5-0-50 ~$ echo "****" && aws ec2 create-snapshot --volume-id vol-07c6122efed356ca >> /tmp/cronlog" > crontab
ec2-user@ip-10-5-0-50 ~$ crontab crontab
ec2-user@ip-10-5-0-50 ~$ aws ec2 describe-snapshots --filters "Name=volume-id,Values=vol-07c6122efed356ca"

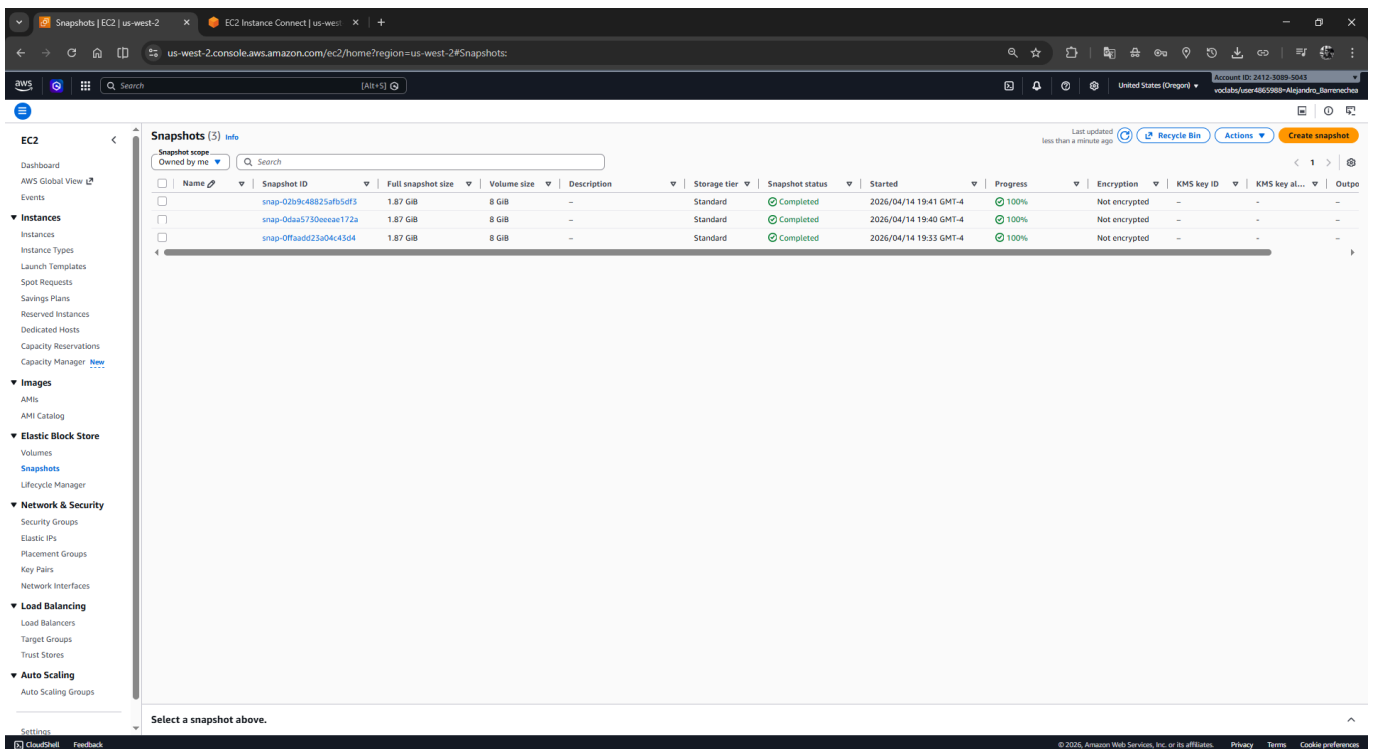
{
  "Snapshots": [
    {
      "StorageTier": "standard",
      "TransferType": "standard",
      "CompletionTime": "2026-04-14T23:34:38.606Z",
      "FullSnapshotSizeInBytes": 204353024,
      "SnapshotId": "snap-0f6ad023a04c4364",
      "VolumeId": "vol-07c6122efed356ca",
      "State": "completed",
      "StartTime": "2026-04-14T23:33:53.402Z",
      "Progress": "100%",
      "OwnerId": "241230895043",
      "Description": "",
      "VolumeSize": 8,
      "Encrypted": false
    }
  ]
}

ec2-user@ip-10-5-0-50 ~$ aws ec2 describe-snapshots --filters "Name=volume-id,Values=vol-07c6122efed356ca"

{
  "Snapshots": [
    {
      "StorageTier": "standard",
      "TransferType": "standard",
      "CompletionTime": "2026-04-14T23:40:34.650Z",
      "FullSnapshotSizeInBytes": 204674464,
      "SnapshotId": "snap-0daa5730eeae172a",
      "VolumeId": "vol-07c6122efed356ca",
      "State": "completed",
      "StartTime": "2026-04-14T23:40:02.416Z",
      "Progress": "100%",
      "OwnerId": "241230895043",
      "Description": "",
      "VolumeSize": 8,
      "Encrypted": false
    },
    {
      "StorageTier": "standard",
      "TransferType": "standard",
      "CompletionTime": "2026-04-14T23:34:38.606Z",
      "FullSnapshotSizeInBytes": 204353024,
      "SnapshotId": "snap-0f6ad023a04c4364",
      "VolumeId": "vol-07c6122efed356ca",
      "State": "completed",
      "StartTime": "2026-04-14T23:33:53.402Z",
      "Progress": "100%",
      "OwnerId": "241230895043",
      "Description": "",
      "VolumeSize": 8,
      "Encrypted": false
    }
  ]
}

ec2-user@ip-10-5-0-50 ~$
```

2. Repite después de unos minutos para observar más snapshots.



Name	Snapshot ID	Full snapshot size	Volume size	Description	Storage tier	Snapshot status	Started	Progress	Encryption	KMS key ID	KMS key a...	Output
snap-02b9c48825afb5df3	snap-02b9c48825afb5df3	1.87 GiB	8 GiB		Standard	Completed	2026/04/14 19:41 GMT-4	100%	Not encrypted	-	-	-
snap-0daa5730eeae172a	snap-0daa5730eeae172a	1.87 GiB	8 GiB		Standard	Completed	2026/04/14 19:40 GMT-4	100%	Not encrypted	-	-	-
snap-0faad623a04c4364	snap-0faad623a04c4364	1.87 GiB	8 GiB		Standard	Completed	2026/04/14 19:33 GMT-4	100%	Not encrypted	-	-	-

🔧 Tarea 2.4: Retener únicamente los dos snapshots más recientes

🔹 Paso 2.4.1: Detener el cron job

```
crontab -r
```

◆ Paso 2.4.2: Revisar el script de retención

```
more /home/ec2-user/snapshotter_v2.py
```

◆ Paso 2.4.3: Verificar los snapshots actuales

```
aws ec2 describe-snapshots --filters "Name=volume-id, Values=vol-07c6122ef1ed356ca" --query 'Snapshots[*].SnapshotId'
```

◆ Paso 2.4.4: Ejecutar el script de retención

```
python3.8 snapshotter_v2.py
```

```

aws
[ec2-user@ip-10-5-0-50 ~]$ more /home/ec2-user/snapshotter_v2.py
#!/usr/bin/env python
import boto3

MAX_SNAPSHOTS = 2 # Number of snapshots to keep
# Create the EC2 resource
ec2 = boto3.resource('ec2')

# Get a list of all volumes
volume_iterator = ec2.volumes.all()

# Create a snapshot of each volume
for v in volume_iterator:
    v.create_snapshot()

# Too many snapshots?
snapshots = list(v.snapshots.all())
if len(snapshots) > MAX_SNAPSHOTS:
    # Delete oldest snapshots, but keep MAX_SNAPSHOTS available
    snap_sorted = sorted([(s.id, s.start_time, s) for s in snapshots], key=lambda k: k[1])
    for s in snap_sorted[-MAX_SNAPSHOTS:]:
        print("Deleting snapshot", s[0])
        s[2].delete()

[ec2-user@ip-10-5-0-50 ~]$ aws ec2 describe-snapshots --filters "Name=volume-id, Values=vol-07c6122ef1ed356ca" --query 'Snapshots[*].SnapshotId'
["snap-02b9c4825a7b5df3",
"snap-02b9c4825a7b5df3",
"snap-0daa5730eeae172a",
"snap-0f1ead23a8c430d4"]

[ec2-user@ip-10-5-0-50 ~]$ python3.8 snapshotter_v2.py
Deleting snapshot snap-0f1ead23a8c430d4
Deleting snapshot snap-0daa5730eeae172a
Deleting snapshot snap-02b9c4825a7b5df3
[ec2-user@ip-10-5-0-50 ~]$ aws ec2 describe-snapshots --filters "Name=volume-id, Values=vol-07c6122ef1ed356ca" --query 'Snapshots[*].SnapshotId'
["snap-02b9c4825a7b5df3",
"snap-0b8997a104b629f93"]

[ec2-user@ip-10-5-0-50 ~]$

```

El script eliminará las snapshots antiguas y dejará solo las dos más recientes.

◆ Paso 2.4.5: Confirmar el resultado

```
aws ec2 describe-snapshots --filters "Name=volume-id, Values=vol-07c6122ef1ed356ca" --query 'Snapshots[*].SnapshotId'
```

El resultado debe mostrar solamente dos IDs de snapshot.

🚩 Tarea 3: Challenge - Sincronizar archivos con Amazon S3

💠 Paso 3.1: Descargar y extraer archivos de ejemplo

1. Conéctate a la instancia **Processor** usando EC2 Instance Connect.
2. Descarga el paquete de archivos:

```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-RSJAWS-3-124627/183-lab-JAWS-managing-storage/s3/files.zip
```

3. Descomprime el archivo:

```
unzip files.zip
```

```

Amazon Linux 2
AL2 End of Life is 2026-06-30.
A newer version of Amazon Linux is available!
Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-5-0-83 ~]$ pwd
/home/ec2-user
[ec2-user@ip-10-5-0-83 ~]$ hostname
ip-10-5-0-83.us-west-2.compute.internal
[ec2-user@ip-10-5-0-83 ~]$ wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-RSJAWS-3-124627/183-lab-JAWS-managing-storage/s3/files.zip
--2026-04-14 23:45:05-- https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-RSJAWS-3-124627/183-lab-JAWS-managing-storage/s3/files.zip
Resolving aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)... 3.5.81.224, 52.92.153.114, 3.5.81.104, ...
Connecting to aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)|3.5.81.224|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 72110 (70K) [application/zip]
Saving to: 'files.zip'

100%[=====] 72,110  --.-K/s  in 0.009s

2026-04-14 23:45:05 (7.43 MB/s) - 'files.zip' saved [72110/72110]

[ec2-user@ip-10-5-0-83 ~]$ unzip files.zip
Archive: files.zip
  inflating: files/file1.txt
  inflating: files/file2.txt
  inflating: files/file3.txt
[ec2-user@ip-10-5-0-83 ~]$
  
```

💠 Paso 3.2: Activar versioning y sincronizar archivos

1. Habilita versioning en tu bucket S3:

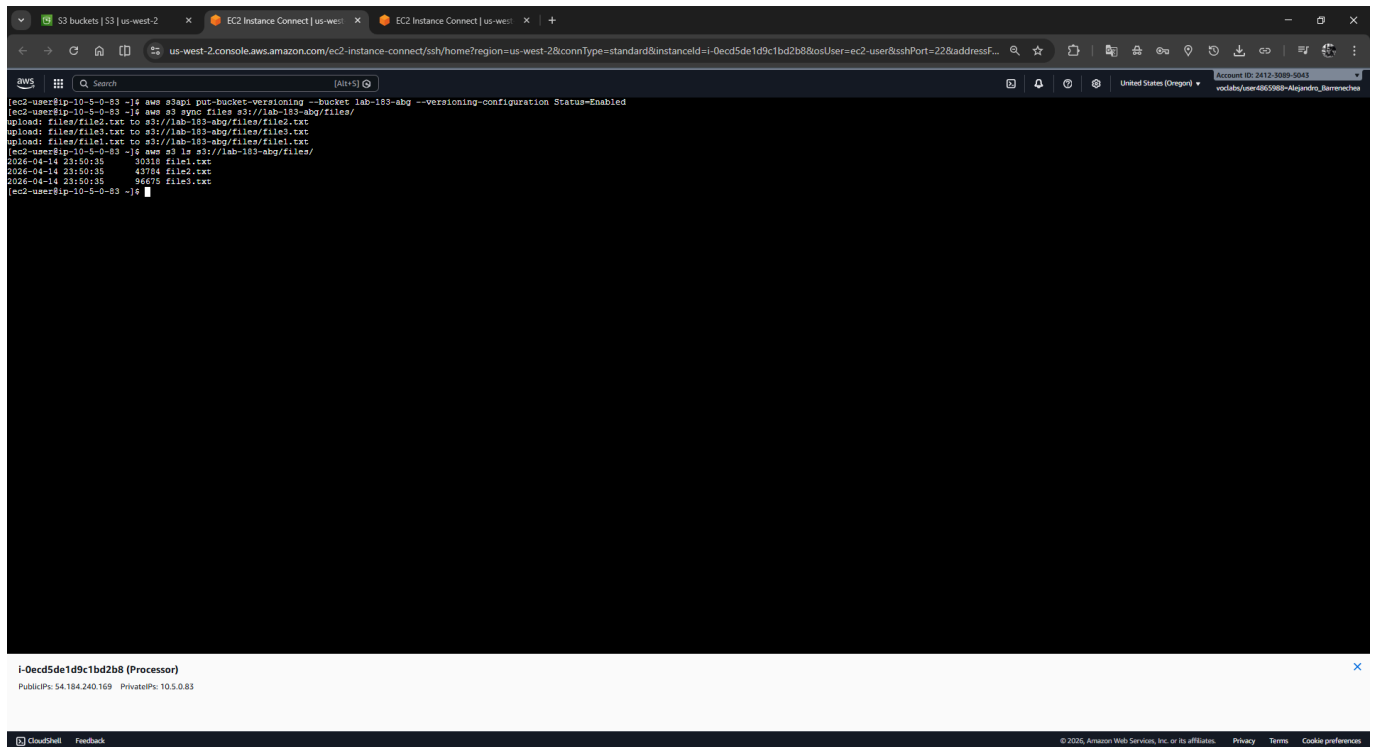
```
aws s3api put-bucket-versioning --bucket lab-183-abg --versioning-configuration Status=Enabled
```

2. Sincroniza la carpeta local **files** con S3:

```
aws s3 sync files s3://lab-183-abg/files/
```

3. Verifica el contenido del bucket:

```
aws s3 ls s3://lab-183-abg/files/
```



The screenshot shows a terminal window with the following commands and output:

```
ec2-user@ip-10-5-0-83 ~$ aws s3api put-bucket-versioning --bucket lab-183-abg --versioning-configuration Status=Enabled
ec2-user@ip-10-5-0-83 ~$ aws s3 sync files s3://lab-183-abg/files/
upload: files/file1.txt to s3://lab-183-abg/files/file1.txt
upload: files/file3.txt to s3://lab-183-abg/files/file3.txt
upload: files/file1.txt to s3://lab-183-abg/files/file1.txt
ec2-user@ip-10-5-0-83 ~$ aws s3 ls s3://lab-183-abg/files/
2026-04-14 23:50:35      30318 file1.txt
2026-04-14 23:50:35      43706 file2.txt
2026-04-14 23:50:35      96675 file3.txt
ec2-user@ip-10-5-0-83 ~$
```

Below the terminal output, a metadata box for the instance `i-0ecd5de1d9c1bd2b8` is visible, showing its public and private IP addresses.

◆ Paso 3.3: Eliminar un archivo local y propagar el cambio

1. Borra un archivo local:

```
rm files/file1.txt
```

2. Sincroniza con eliminación remota:

```
aws s3 sync files s3://lab-183-abg/files/ --delete
```

3. Verifica el archivo eliminado en el bucket:

```
aws s3 ls s3://lab-183-abg/files/
```

```
aws s3api put-bucket-versioning --bucket lab-183-abg --versioning-configuration Status=Enabled
[ec2-user@ip-10-5-0-83 ~]$ aws s3 sync files s3://lab-183-abg/files/
upload: files/file2.txt to s3://lab-183-abg/files/file2.txt
upload: files/file1.txt to s3://lab-183-abg/files/file1.txt
[ec2-user@ip-10-5-0-83 ~]$ aws s3 ls s3://lab-183-abg/files/
2026-04-14 23:50:35      30318 file1.txt
2026-04-14 23:50:35      43784 file2.txt
2026-04-14 23:50:35      96675 file3.txt
[ec2-user@ip-10-5-0-83 ~]$ rm files/file1.txt
delete: s3://lab-183-abg/files/file1.txt
[ec2-user@ip-10-5-0-83 ~]$ aws s3 ls s3://lab-183-abg/files/
2026-04-14 23:50:35      43784 file2.txt
2026-04-14 23:50:35      96675 file3.txt
[ec2-user@ip-10-5-0-83 ~]$
```

i-0ecd5de1d9c1bd2b8 (Processor)
PublicPps: 54.184.240.169 PrivatePps: 10.5.0.83

◆ Paso 3.4: Recuperar el archivo eliminado usando versioning

1. Lista las versiones de **file1.txt**:

```
aws s3api list-object-versions --bucket lab-183-abg --prefix files/file1.txt
```

```
[ec2-user@ip-10-5-0-83 ~]$ aws s3api list-object-versions --bucket lab-183-abg --prefix files/file1.txt
{
  "Versions": [
    {
      "ETag": "\"b7fb2b775023e60be1ebc332496f8409\"",
      "ChecksumAlgorithm": "CRC32",
      "ChecksumType": "FULL_OBJECT",
      "Size": 30318,
      "StorageClass": "STANDARD",
      "Key": "files/file1.txt",
      "VersionId": "QakwCAZUJ3cnoc2XIHUloogBvaySRfy",
      "IsLatest": false,
      "LastModified": "2026-04-14T23:50:35.000Z",
      "Owner": {
        "ID": "33493a83a0f9ea4833e586d67b7039b23aabe4220fb804f090b00070af4f0c4d"
      }
    },
    {
      "Owner": {
        "ID": "33493a83a0f9ea4833e586d67b7039b23aabe4220fb804f090b00070af4f0c4d"
      },
      "Key": "files/file1.txt",
      "VersionId": "Dagf4l19oe5jwGMLJANuYgSc.SwaVU.E",
      "IsLatest": true,
      "LastModified": "2026-04-14T23:51:25.000Z"
    }
  ],
  "DeleteMarkers": [
    {
      "Owner": {
        "ID": "33493a83a0f9ea4833e586d67b7039b23aabe4220fb804f090b00070af4f0c4d"
      },
      "Key": "files/file1.txt",
      "VersionId": "Dagf4l19oe5jwGMLJANuYgSc.SwaVU.E",
      "IsLatest": true,
      "LastModified": "2026-04-14T23:51:25.000Z"
    }
  ],
  "RequestCharged": null,
  "Prefix": "files/file1.txt"
}
[ec2-user@ip-10-5-0-83 ~]$ ^C
[ec2-user@ip-10-5-0-83 ~]$ aws s3api get-object --bucket lab-183-abg --key files/file1.txt --version-id QakwCAZUJ3cnoc2XIHUloogBvaySRfy files/file1.txt
{
  "AcceptRanges": "bytes",
  "LastModified": "Tue, 14 Apr 2026 23:50:35 GMT",
  "ContentLength": 30318,
  "ETag": "\"b7fb2b775023e60be1ebc332496f8409\"",
  "ChecksumMD5": "qnpPQm=",
  "ChecksumType": "FULL_OBJECT",
  "VersionId": "QakwCAZUJ3cnoc2XIHUloogBvaySRfy",
  "ContentType": "text/plain",
  "ServerSideEncryption": "AES256",
  "Metadata": {}
}
[ec2-user@ip-10-5-0-83 ~]$ ls files
file1.txt file2.txt file3.txt
[ec2-user@ip-10-5-0-83 ~]$
```

i-0ecd5de1d9c1bd2b8 (Processor)
PublicPps: 54.184.240.169 PrivatePps: 10.5.0.83

2. Descarga esa versión localmente:

```
aws s3api get-object --bucket lab-183-abg --key files/file1.txt --version-id
QzkuCAZUJjcnoc2XiHUIoogcBvaySRFy files/file1.txt
```

3. Verifica que el archivo volvió a estar local:

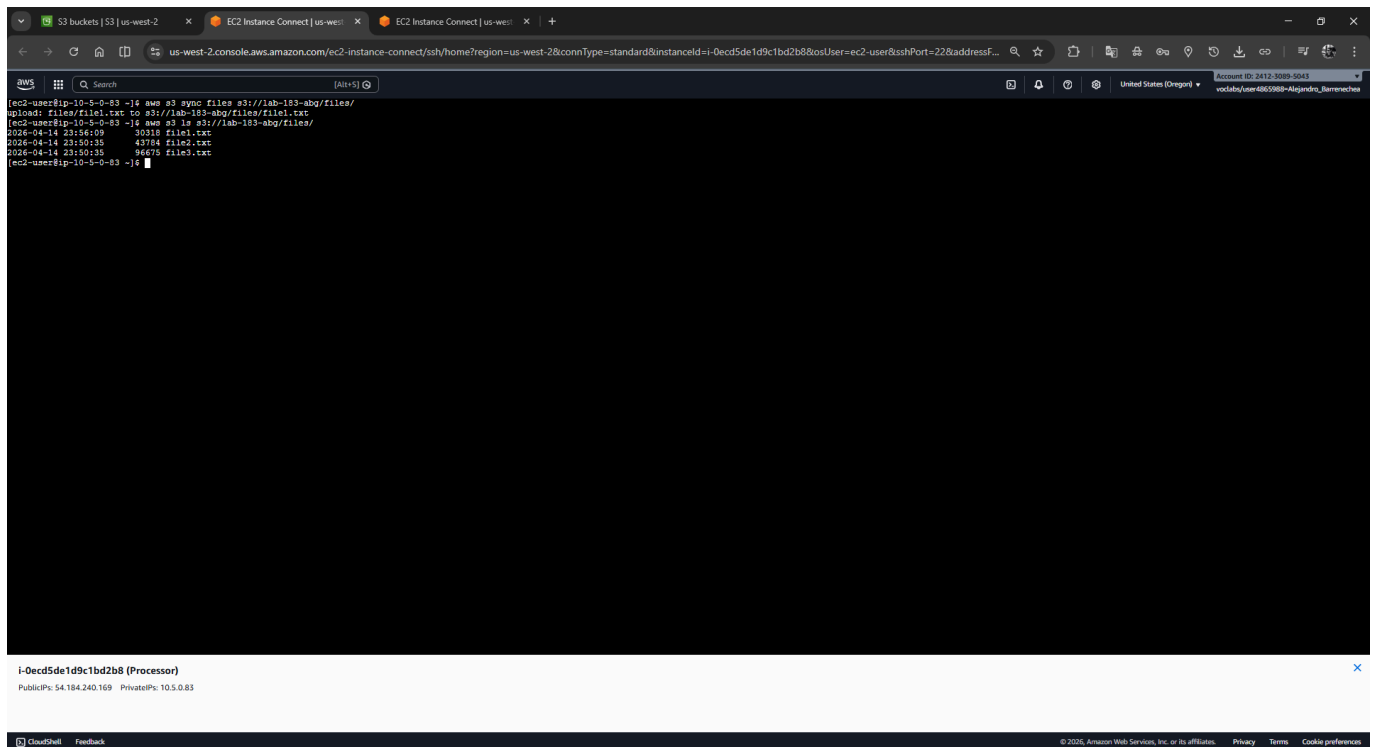
```
ls files
```

4. Re-sincroniza la carpeta local con S3:

```
aws s3 sync files s3://lab-183-abg/files/
```

5. Confirma que el nuevo objeto fue cargado:

```
aws s3 ls s3://lab-183-abg/files/
```



```
aws s3 sync files s3://lab-183-abg/files/
aws s3 ls s3://lab-183-abg/files/
```

The screenshot shows a terminal window with the following output:

```
ec2-user@ip-10-5-0-83 ~$ aws s3 sync files s3://lab-183-abg/files/
upload: files/file1.txt to s3://lab-183-abg/files/file1.txt
ec2-user@ip-10-5-0-83 ~$ aws s3 ls s3://lab-183-abg/files/
2026-04-14 23:54:09    30310 file1.txt
2026-04-14 23:50:35    43784 file2.txt
2026-04-14 23:50:35    96675 file3.txt
ec2-user@ip-10-5-0-83 ~$
```

💡 Respuestas Analíticas a Preguntas Clave

¿Por qué usar AWS CLI en lugar de solo consola?

AWS CLI permite automatizar tareas repetitivas, ejecutar scripts, y reproducir procesos exactamente con comandos. En este laboratorio, AWS CLI es esencial para cron, snapshots y sincronización de archivos.

¿Por qué activar versioning en S3?

Versioning permite recuperar objetos eliminados o revertir cambios. Una sincronización con `--delete` crea un marcador de borrado, pero la versión previa del objeto sigue disponible.

¿Por qué detener Processor antes de crear el primer snapshot?

Se detiene para garantizar consistencia de datos en el volumen. Un snapshot en caliente puede ser válido, pero detener la instancia elimina el riesgo de escribir datos durante la copia.

¿Qué hace el script `snapshotter_v2.py`?

El script ordena snapshots por fecha y elimina las copias más antiguas, manteniendo solo las dos más recientes. Esto minimiza costos y mantiene un historial útil.

¿Por qué usar `aws s3 sync` con `--delete`?

Porque sincroniza el bucket con la estructura local. Si un archivo se borra localmente, `--delete` elimina automáticamente su copia en S3.



Conclusión

Has completado el flujo completo de gestión de almacenamiento AWS:

- Creaste un bucket S3 y configuraste permisos con IAM
- Tomaste snapshots de un volumen EBS con AWS CLI
- Programaste y detuviste snapshots automatizados con cron
- Ejecutaste un script para retener solo los snapshots más recientes
- Sincronizaste datos locales con S3 usando `aws s3 sync`
- Verificaste y recuperaste un archivo eliminado con S3 versioning

Este laboratorio demuestra cómo combinar EBS, EC2, S3 y AWS CLI para una administración de almacenamiento segura, automatizada y recuperable.